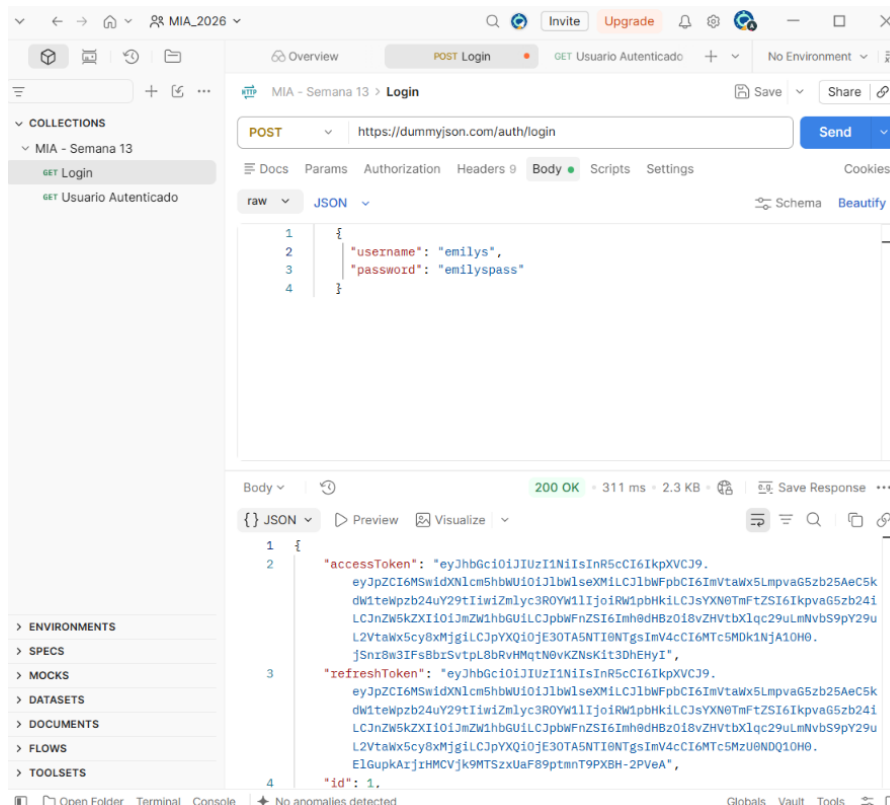
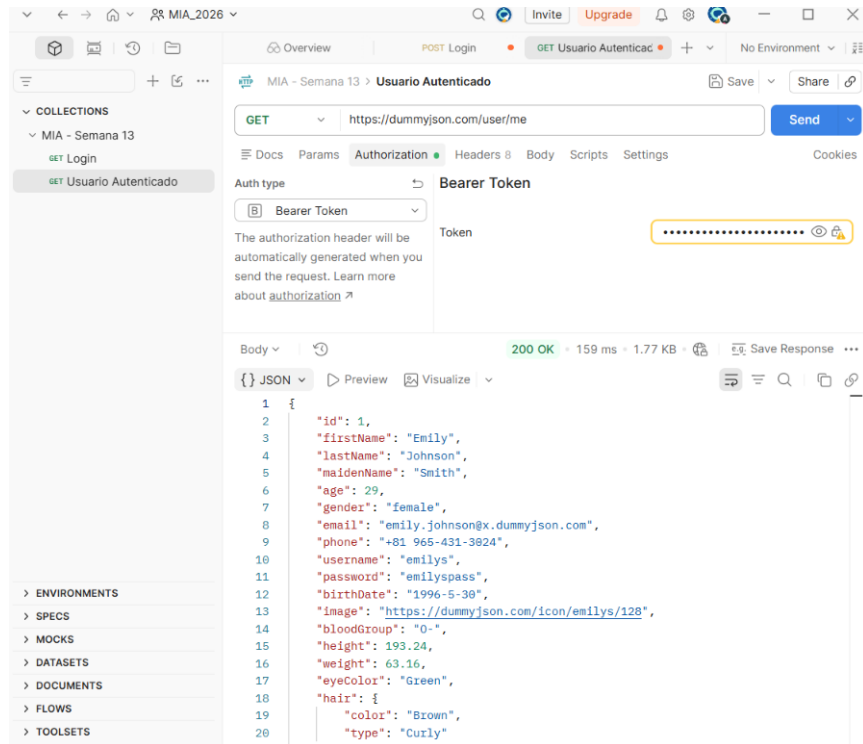




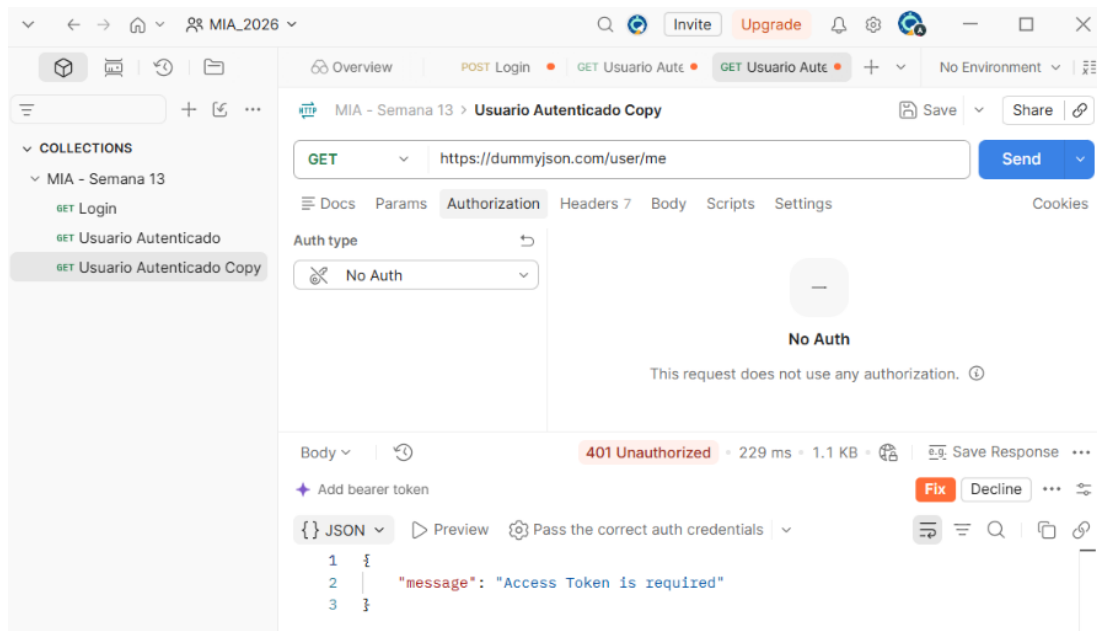
2. Utilizar token

- Cree una segunda solicitud llamada: Usuario autenticado.
- Seleccione GET.
- Utilice: <https://dummyjson.com/user/me>
- En Authorization seleccione Bearer Token.
- Copie el accessToken obtenido en la solicitud Login.
- Ejecute la solicitud.
- Verifique que la API devuelve la información del usuario autenticado.



3. Comprobar qué ocurre sin autenticación

- Duplique la solicitud Usuario autenticado.
- Quite el token.
- Ejecute nuevamente la solicitud.
- Compare la respuesta con la solicitud autenticada.
- Registre el código HTTP y explique la diferencia.



4. Evidencia

- Captura de Login mostrando la respuesta.
- Captura de la solicitud autenticada.
- Captura de la prueba sin token.
- Indique qué función cumple el Bearer Token.
 - El Bearer Token es una credencial temporal que el servidor entrega después de un login exitoso. El cliente la envía en cada solicitud dentro del encabezado Authorization: Bearer <token>. Así el servidor identifica y autoriza al usuario sin que este tenga que enviar su usuario y contraseña en cada petición. Quien "porta" (bearer) el token obtiene acceso, por eso hay que protegerlo.

La documentación de DummyJSON muestra el flujo de login, la obtención del accessToken y su envío mediante el encabezado Authorization: Bearer.

Parte 2 – Proteger la Connection String de Azure Storage

Utilice como base el programa C# desarrollado anteriormente para conectarse a Azure Storage. No se requiere crear nuevamente el Storage Account

5. Localizar la Connection String

- Abra el proyecto C# de Azure Storage.
- En el Explorador de soluciones, abra el archivo donde actualmente está escrita la Connection String. En un proyecto sencillo normalmente será Program.cs.
- Busque la variable que contiene la Connection String.
- Identifique el valor que comienza con información como DefaultEndpointsProtocol=...

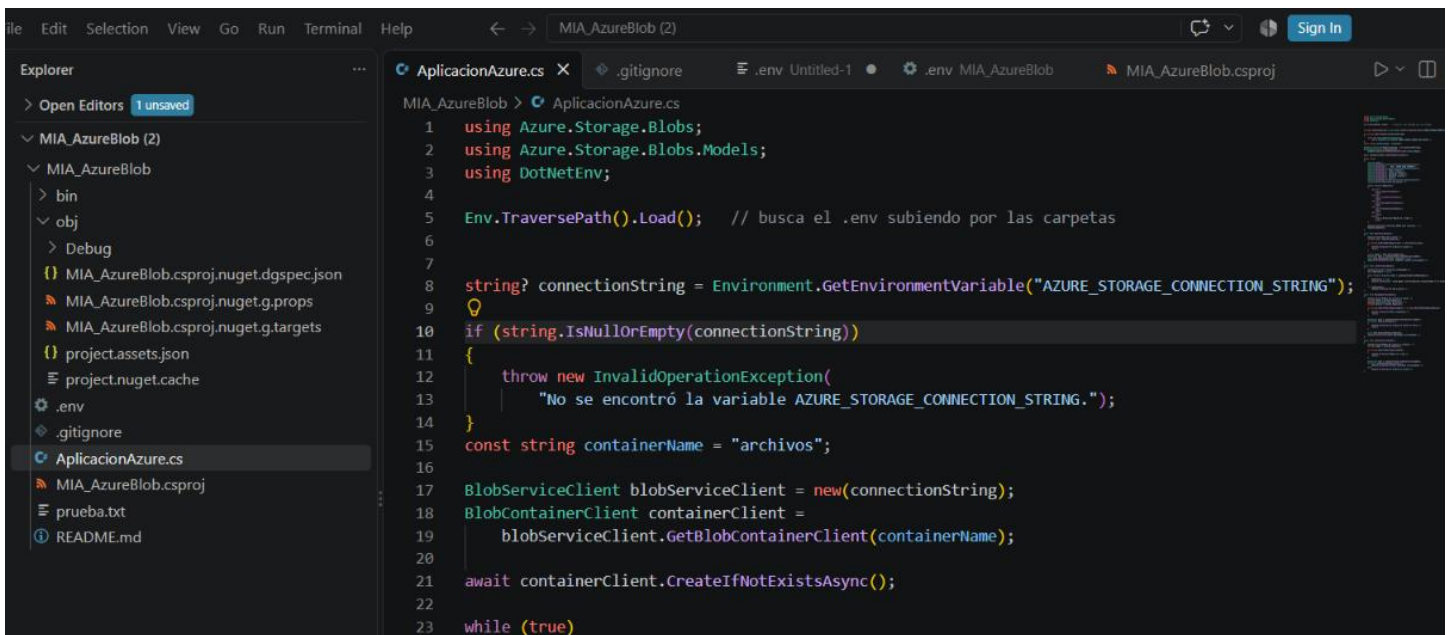
6. Reemplazar la Connection String

Reemplace el valor escrito directamente en el programa por una lectura de una variable de entorno llamada: **AZURE_STORAGE_CONNECTION_STRING**

La idea es que el programa siga utilizando la misma variable para conectarse a Azure, pero ahora el valor será obtenido desde el entorno y no desde el código

7. Crear la variable para ejecutar localmente

- En la carpeta principal del proyecto, cree un archivo llamado .env.
- Dentro del archivo agregue la variable:
AZURE_STORAGE_CONNECTION_STRING=TU_CONNECTION_STRING
- Reemplace SU_CONNECTION_STRING por la Connection String de su Storage Account.
- Verifique que el programa lea la variable desde el .env y ejecute nuevamente el programa.
- Compruebe que el programa continúa funcionando correctamente.
- No publique el archivo .env en GitHub**, ya que contiene la credencial real.



The screenshot shows the Visual Studio IDE with the following details:

- Explorer:** The file explorer on the left shows the project structure for 'MIA_AzureBlob (2)'. The file 'AplicacionAzure.cs' is selected.
- AplicacionAzure.cs:** The code in the main editor shows the following C# code:

```
1 using Azure.Storage.Blobs;
2 using Azure.Storage.Blobs.Models;
3 using DotNetEnv;
4
5 Env.TraversePath().Load(); // busca el .env subiendo por las carpetas
6
7
8 string? connectionString = Environment.GetEnvironmentVariable("AZURE_STORAGE_CONNECTION_STRING");
9
10 if (string.IsNullOrEmpty(connectionString))
11 {
12     throw new InvalidOperationException(
13         "No se encontró la variable AZURE_STORAGE_CONNECTION_STRING.");
14 }
15 const string containerName = "archivos";
16
17 BlobServiceClient blobServiceClient = new(connectionString);
18 BlobContainerClient containerClient =
19     blobServiceClient.GetBlobContainerClient(containerName);
20
21 await containerClient.CreateIfNotExistsAsync();
22
23 while (true)
```
- .env:** The file explorer shows a '.env' file, which is the file created for local execution.

```

PS C:\Users\Angela\Desktop\url-mia-2026\Semana_12\Lab2\MIA_AzureBlob (2)\MIA_AzureBlob> git check-ignore -v .env
Semana_12\Lab2\MIA_AzureBlob (2)\MIA_AzureBlob/.gitignore:1:.env      .env
PS C:\Users\Angela\Desktop\url-mia-2026\Semana_12\Lab2\MIA_AzureBlob (2)\MIA_AzureBlob> git status -uall
On branch main
Your branch is up to date with 'origin/main'.

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .env.example
        .gitignore
        AplicacionAzure.cs
        MIA_AzureBlob.csproj
        README.md
        prueba.txt

nothing added to commit but untracked files present (use "git add" to track)
PS C:\Users\Angela\Desktop\url-mia-2026\Semana_12\Lab2\MIA_AzureBlob (2)\MIA_AzureBlob>

```

8. Crear el Secret en Github

- Ingrese al repositorio de su proyecto en GitHub.
- Seleccione Settings.
- En el menú lateral seleccione Secrets and variables.
- Seleccione Actions.
- Seleccione New repository secret.
- En Name escriba: AZURE_STORAGE_CONNECTION_STRING
- En Secret coloque la Connection String de Azure Storage.
- Seleccione Add secret.

The screenshot shows the GitHub repository settings for 'url-mia-2026'. The 'Settings' tab is selected, and the 'Actions secrets and variables' section is expanded. The 'Repository secrets' tab is active, showing a table with one secret named 'AZURE_STORAGE_CONNECTION_STRING'. The 'Environment secrets' section shows 'This environment has no secrets.' and a 'Manage environment secrets' button. A 'New repository secret' button is visible in the top right corner of the secrets section.

Entrega

Dentro del repositorio de GitHub, cree la carpeta semana-13/lab2/ y coloque en ella un PDF con la solución del laboratorio.

Incluya también los archivos de respaldo generados durante la práctica, así como los pantallazos que evidencien las pruebas realizadas.

Enviar link de github a plataforma de la universidad.